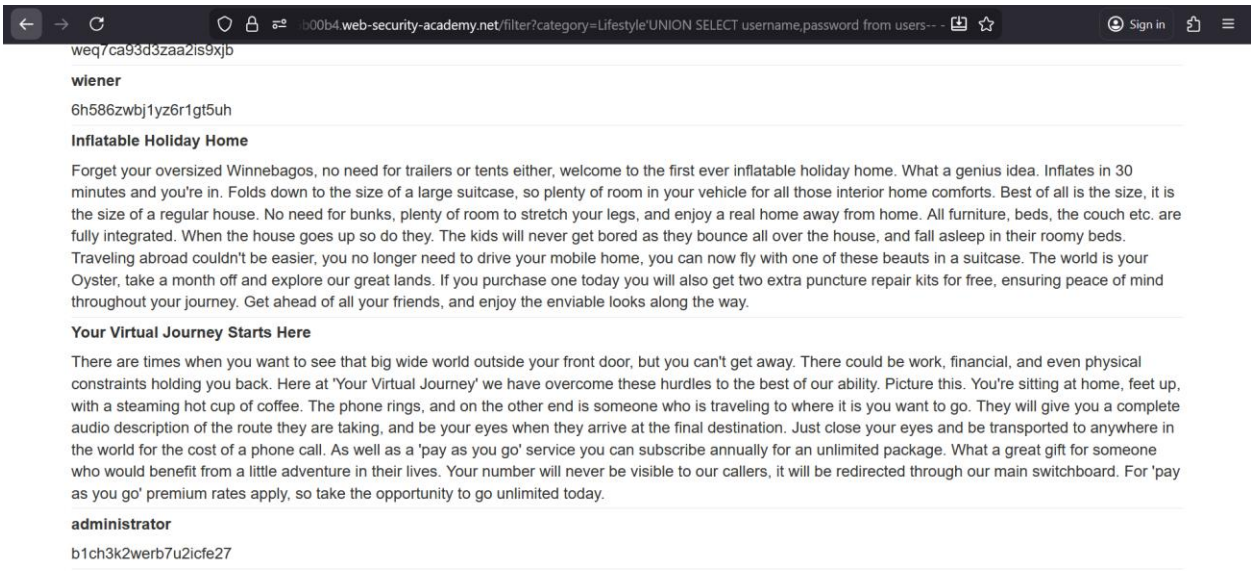
Title: determined the website that have SQL injection vulnerability to get the username and password of the users

SQL Injection forces a web application to execute unintended SQL commands by injecting malicious input into queries. Failures typically lead to unauthorized access to the database, exposure of sensitive data, deletion or modification of records, or full compromise of the backend system.

Reproduction Steps/Point of view:

1.visit https://0a46007c038fd3e9804b2102007a0017.web-security-academy.net/

2. perform a malicious search Pets and write('UNION SELECT username,password from users-- -)



3.now we can access to the admin account

Impact Statement:

Successful exploitation of this SQL Injection vulnerability allows an unauthenticated attacker to perform a UNION-based attack to directly retrieve the contents of the `users` table, exposing all usernames and passwords stored within the database. This results in a complete compromise of the application's authentication mechanism, granting the attacker full access to all user accounts including the `administrator` account.

This vulnerability poses a **Critical** risk to the organization as it enables direct extraction of user credentials without any prior authentication, bypassing all application-level access controls. The attacker gains immediate ability to access, manipulate, or exfiltrate sensitive user data, which may include Personally Identifiable Information (PII) and other confidential records.

Furthermore, this level of access severely impacts all three pillars of the CIA Triad:

- **Confidentiality:** All stored usernames and passwords are fully exposed to the attacker, resulting in complete loss of credential confidentiality across all user accounts.
- **Integrity:** With administrator-level access, the attacker may modify, insert, or delete any database records, compromising the accuracy and trustworthiness of all stored data.
- **Availability:** The attacker could leverage administrator privileges to disrupt application functionality, delete critical records, or cause service downtime.

The direct extraction of administrator credentials represents a worst-case scenario, as it grants the attacker the highest level of privilege within the application. This may further enable **lateral movement** across connected systems and infrastructure, significantly

amplifying the overall impact. This breach may also result in severe **regulatory and legal consequences** under data protection frameworks such as GDPR, as the exposure of user credentials constitutes a reportable data breach.

Mitigation:

To remediate this SQL Injection vulnerability exploited via a UNION-based attack to extract credentials directly from the `users` table, it is strongly recommended to implement **Parameterized Queries (Prepared Statements)** for all database queries involving user-supplied input such as the `category` parameter. This prevents attackers from injecting additional SQL clauses like `UNION SELECT username, password FROM users` to retrieve data from unintended tables.

Enforce **strict input validation and whitelisting** on the `category` parameter, accepting only predefined expected values and rejecting any input containing SQL keywords or special characters such as `'`, `UNION`, `SELECT`, or `--`.

Ensure that **passwords are never stored in plaintext** within the database. All passwords must be hashed using strong hashing algorithms such as `bcrypt` or `Argon2` with proper salting, ensuring that even if credentials are extracted from the database, they cannot be used directly without cracking.

Apply the **principle of least privilege** to the database account used by the application, ensuring it has no unnecessary read access to sensitive tables such as `users`. This limits the attacker's ability to extract credential data even if injection is partially successful.

Implement **multi-factor authentication (MFA)** for all user accounts, particularly privileged accounts such as `administrator`, adding an additional layer of defense against credential-based login attempts even if credentials are compromised.

Ensure that **detailed database error messages** are never returned to the end user, as they can reveal database structure and assist further exploitation attempts.

Finally, deploy a **Web Application Firewall (WAF)** to detect and block common UNION-based SQL Injection payloads before they reach the application, providing an additional defensive layer alongside the core fixes above.